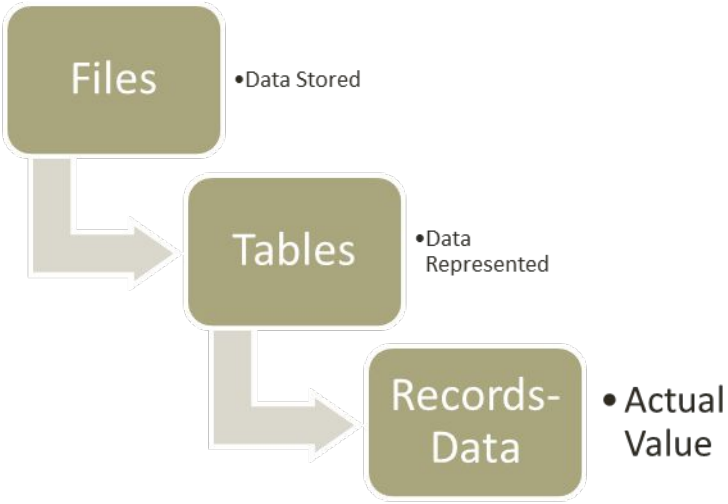
Chapter 1: Introduction

Chapter 1: Introduction

- -Introduction to Database Management System
- Purpose of Database System, View of Data.
- Data Models, Relational Model, Network Model, Hierarchical Model, E-R Model, Comparison between models.
- Data Abstraction.
- Instance and schemas.
- Database Languages-Data Manipulation Languages, Data Definition Languages.
- Database Users and Administrator-Database Users, Database Administrator.

Database Management System (DBMS)

- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use
  - Goal-store/retrieve the data
  - DBMS-Structure and manipulation.



- Eg:Store records of Book
- 101 DBMS KJSP Sem-IV 200
- Book Code Name Inst Course offered price

- Database Applications:
  - Banking: all transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - telecommunication
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
- Databases touch all aspects of our lives

Database Functions

1.Data Definition-

- Define Structure of Data
- Constraints/Conditions

2. Data Manipulation

- insert/,update,delete
- Planned/unplanned

3.Data Security and integrity

- **protection of data** against unauthorized access-AEM.
- Data integrity refers to the **accuracy, consistency, and reliability** of data.

Feature	Data Security	Data Integrity
Primary Goal	Protection from unauthorized access	Accuracy and consistency of data
Focus	External and internal threats (hackers, leaks)	Validity of the data content and structure
Methods	Encryption, Passwords, Firewalls	Primary Keys, Foreign Keys, Constraints
Threats	Data breaches, unauthorized modification	Human error, software bugs, hardware failure

4.Data Recovery and concurrency-

- restoring the database to a correct and consistent state
- **Concurrency control** is a mechanism that manages simultaneous access to shared data by multiple transactions in a multi-user environment.

5.Performance

Purpose of Database Systems

- In the early days, database applications were built directly on top of file systems
- Drawbacks of using file systems to store data:

1. Data redundancy and inconsistency
    - Different structures(Files)
    - Different programming languages
    - Duplication of data-Eg
    - Redundancy-High Cost and More storage
    - Data inconsistency.

2. Difficulty in accessing data
    - Need to write a new program to carry out each new task

- Data isolation —is a fundamental property of database transactions in a DBMS that guarantees that multiple concurrent transactions run independently and do not interfere with each other

- Integrity problems
  - Integrity constraints problem
  - Hard to add new constraints or change existing ones

Purpose of Database Systems (Cont.)

- Drawbacks of using file systems (cont.)
  - Atomicity of updates
    - Failures may leave database in an inconsistent state with partial updates carried out
    - Example: Transfer of funds from one account to another should either complete or not happen at all
  - Concurrent access by multiple users
    - Concurrent accessed needed for performance
    - Uncontrolled concurrent accesses can lead to inconsistencies
      - Example: Two people reading a balance and updating it at the same time
  - Security problems
    - Hard to provide user access to some, but not all, data
- Database systems offer solutions to all the above problems

Data Abstraction

- Data abstraction is the process of hiding unwanted and irrelevant details from the end user.
- data secure- unauthorized access
- hides -implementation details.
- Eg:Shopping

Levels of Abstraction-

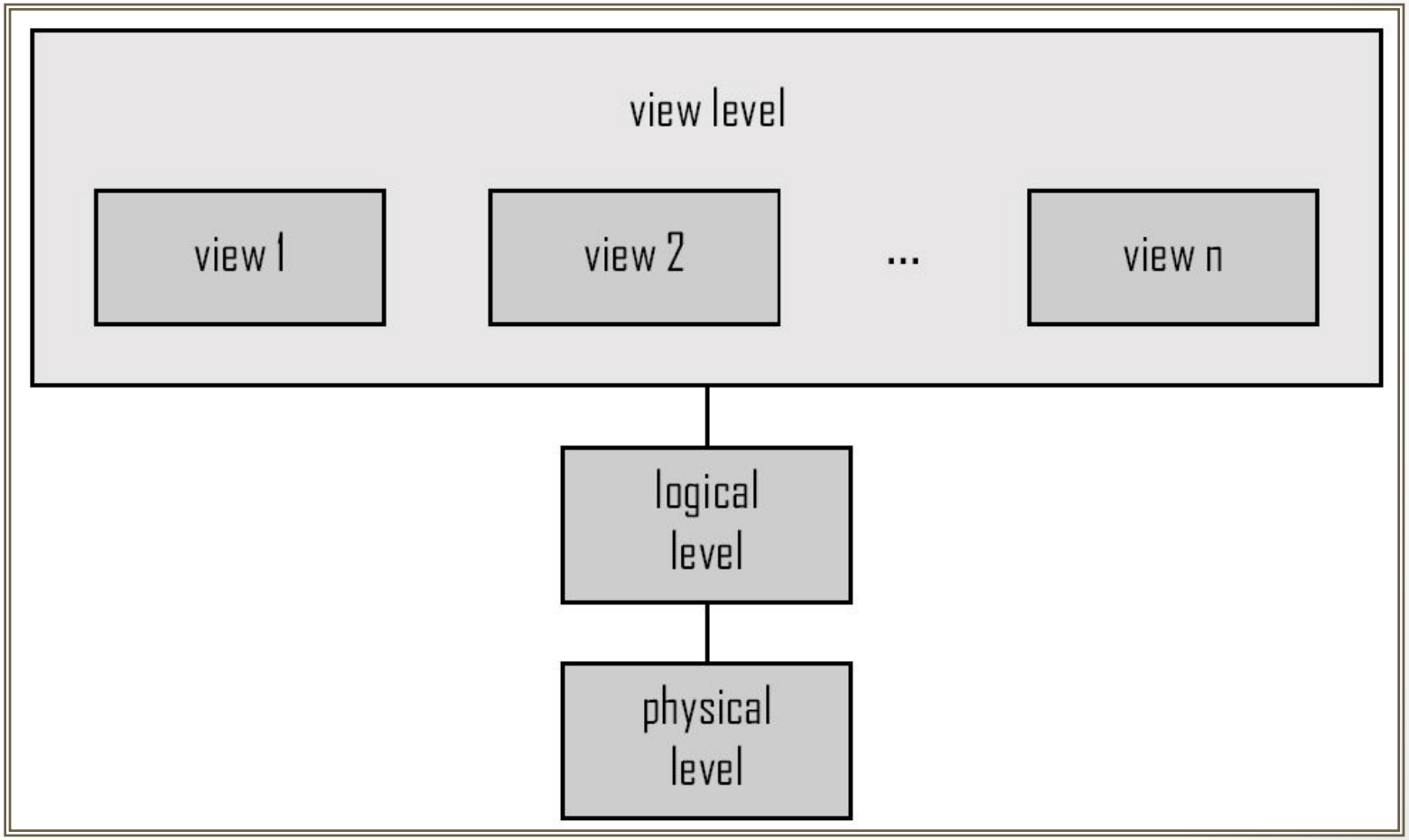
- **Physical level:** describes how a record (e.g., customer) is stored.
- data structures
- methods to access data in database
- complex to understand
- kept hidden from user.
- DBA

- **Logical level:** describes data stored in database, and the relationships among the data.
- Programmers generally work at this level and depending on data, structure of tables, relationships and their constraints is decided at this level.

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

View of Data

An architecture for a database system



Instances and Schemas

- Similar to types and variables in programming languages
- **Schema –** The overall design of the database is called database schema
- the logical structure of the database
- Schema will not be changed frequently
- **Physical schema** − It is a database design at the physical level.It is hidden below the logical schema and can be changed easily without affecting the application programs.
- **Logical schema** − It is a database design at the logical level. Programmers construct applications using logical schema.
- **External** − It is schema at view level. It is the highest level of a schema which defines the views for end users.

Example

A database schema for a person will have fields for name, email, phone and address as shown below –

Person

Name	Email	Phone no

- **Instance** – the actual content of the database at a particular point in time
- it is a dynamic value which keeps on changing.
- Eg: Person Table

The major differences between schema and instance are as follows

<table>
<tr><th>Database Schema</th><th>Database Instance</th></tr>
<tr><td>It is the definition of the database or it is defined as the description of the database.</td><td>It is a snapshot of a database at a specific moment.</td></tr>
<tr><td>It rarely changes.</td><td>It changes frequently.</td></tr>
<tr><td>Example" We take two tables emp table and dept table.Emp
Id
Name
Salary
dept
Dept
Dept_id
dname
Emp and dept both called as schemas
It gives database definition</td><td>At a moment, what is the value of the database schema is called instance.At t=8 A.M
<table><tr><th>Empid</th><th>name</th><th>salary</th><th>did</th></tr><tr><td>1</td><td>A</td><td>5000</td><td>d1</td></tr><tr><td>2</td><td>B</td><td>2000</td><td>d2</td></tr></table>At t=9 A.M
3
C
3000
d3
Empid 1 and 2 are called as Instance 1
At time 9 A.M instance 2 changes</td></tr>
<tr><td>This corresponds to the variable declaration of a programming language.</td><td>The value of the variable in a program at a point in time corresponds to an instance of the database schema.</td></tr>
</table>

Data Models

- Data models define how data is connected to each other and how they are processed and stored inside the system.
- Hierarchical database model
- Relational model
- Network model
- Entity-relationship model

Relational Model

- The most popular data model in DBMS is the Relational Model.
- Data is stored in tables called **relations**.
- Relations can be normalized.
- In normalized relations, values saved are atomic values.
- Each row in a relation contains a unique value.
- Each column in a relation contains values from a same domain.
- Some of the most popular databases are based on this database model. For example, **Oracle**, **MySQL**, etc.
- **Advantages of the Relational Model**
- It's simple and easy to implement.
- Poplar database software is available for this database model.
- It supports SQL using which you can easily query the data.

attributes

column

SID	SName	SAge	SClass	SSection
1101	Alex	14	9	A
1102	Maria	15	9	A
1103	Maya	14	10	B
1104	Bob	14	9	A
1105	Newton	15	10	B

tuple

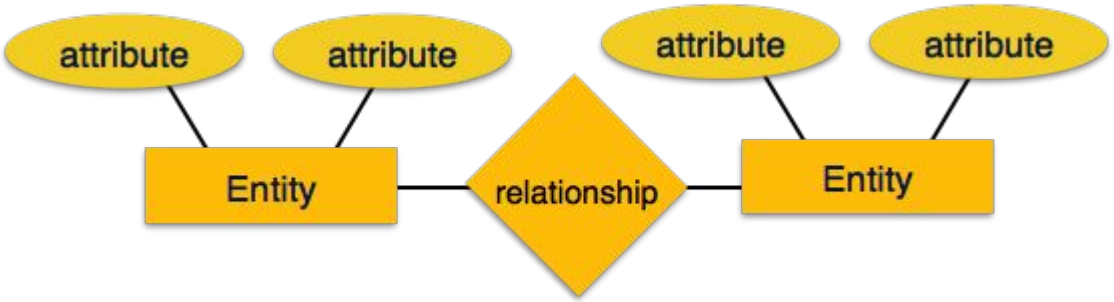
table (relation)

Entity-relationship Model

- In this database model, relationships are created by dividing objects of interest into entities and their characteristics into attributes.
- Different entities are related using **relationships**.
- ER Models are defined to represent the relationships in pictorial form to make it easier for different stakeholders to understand.
- This model is good to design a database, which can then be turned into tables in a relational model

ER Model is based on –

- Entities and their attributes.
- Relationships among entities.



- Entity – An entity in an ER Model is a real-world entity having properties called attributes. Every attribute is defined by its set of values called domain. For example, in a school database, a student is considered as an entity. Student has various attributes like name, age, class, etc.

- Relationship – The logical association among entities is called relationship. Relationships are mapped with entities in various ways. Mapping cardinalities define the number of association between two entities.

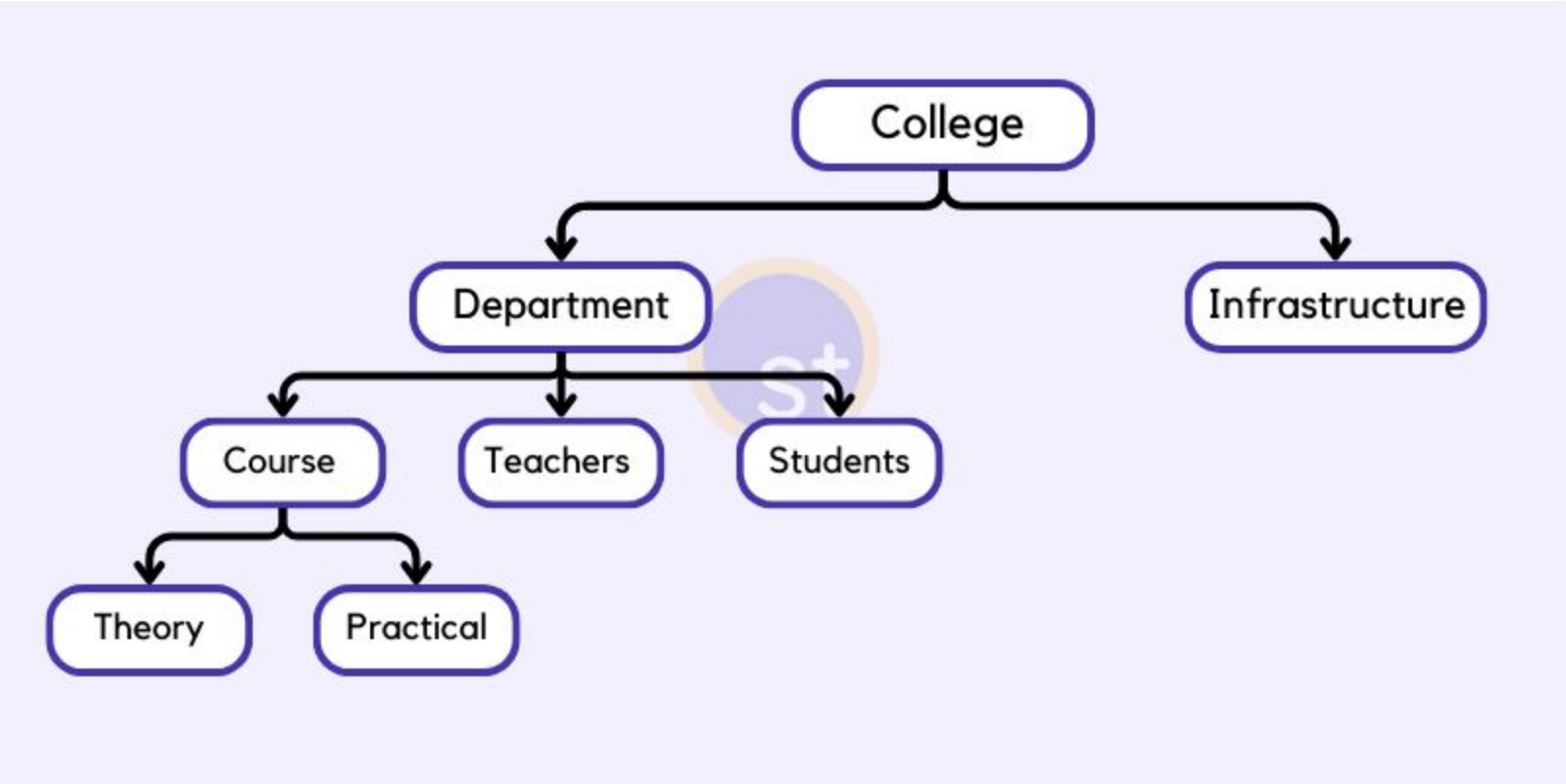
- Mapping cardinalities –

- one to one
- one to many
- many to one
- many to many

- Advantages of the ER Model
- It is easy to understand and design.
- Using the ER model we can represent data structures easily.

Hierarchical Model

- The hierarchical database model organizes data into a **tree-like structure**, with a **single root**, to which all the other data is linked.
- The hierarchy starts from the **Root** data, and expands like a tree, adding **child** nodes to the **parent** nodes.
- In this model, a child node will only have a **single parent node**.
- Data is organized into a tree-like structure with a **one-to-many relationship** between two different types of data, for example, one **department** can have many **courses**, many **teachers**, and of course many **students**(like shown in the diagram below).

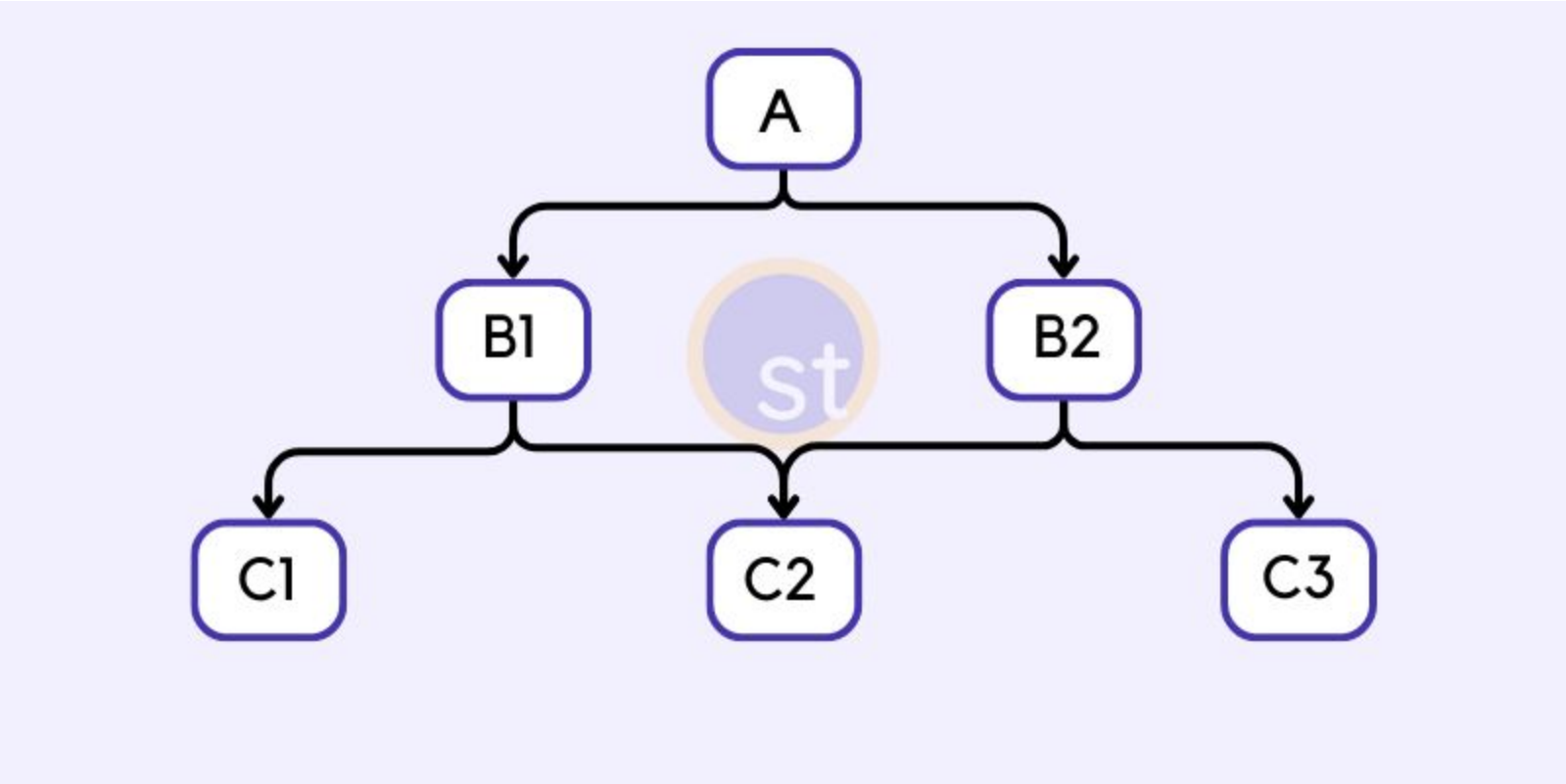

College
Department
Infrastructure
Course
Teachers
Students
Theory
Practical

Advantages/Disadvantages of the Hierarchical Model

- Because it has one-to-many relationships between different types of data so it is easier and fast to fetch the data.
- But the Hierarchical model is less flexible.
- accessing complicated data structures may be challenging.
- redundant data storage, which might cause anomalies and inconsistencies.

Network Model

- The Network Model is an extension of the Hierarchical model.
- In this model, data is organized more like a **graph**, and allowed to have *more than one parent node*.
- In the network database model, data is **more related** as more relationships are established in this database model.
- Also, as the data is more related, hence **accessing the data is also easier** and **fast**.
- This database model uses many-to-many data relationships.
- This was the most widely used database model before Relational Model was introduced.
- The implementation of the Network model is complex, and it's very difficult to maintain it.
- The Network model is difficult to modify also.


A
B1
B2
C1
C2
C3

Advantages/Disadvantages of the Network Model

- It supports complex relationships
- It allows more flexibility
- Increased complexity in database design and management.
- requires complex programming in order to manage and work with data.

Hierarchical Data Model	Network Data Model	Relational Data Model
In this model, to store data hierarchy method is used. It is the oldest method and not in use today.	It organizes records to one another through links or pointers.	It organizes records in the form of table and relationship between tables are set using common fields.
To organize records, it uses tree structure.	It organizes records in the form of directed graphs.	It organizes records in the form of tables.
It implements 1:1 and 1:n relations.	In addition to 1:1 and 1:n it also implements many to many relationships.	In addition to 1:1 and 1:n it also implements many to many relationships.
Pointers are used to establish relationships among records physically.	A linked list is used to establish a relationship among records physically.	The logical representation is used with rows and columns to depict relationship among records.
Insertion anomaly exits in this model i.e. child node cannot be inserted without the parent node.	There is no insertion anomaly.	There is no insertion anomaly.
Deletion anomaly exists in this model i.e. it is difficult to delete the parent node.	There is no deletion anomaly.	There is no deletion anomaly.
Update leads to inconsistency problems because of the existence of multiple instances of a child record.	No such problem as only one instance of records exist.	Updating a record is easy and simple with the process of normalization, the redundant data gets removed.
This model lacks data independence.	There is partial data independence in this model.	This model provides data independence.
No such facility for querying database is supported.	No such facility for querying database is supported.	SQL-based declarative querying is supported.

Database Languages

- These commands are divided into categories such as
- Data Definition Language (**DDL**),
- Data Manipulation Language (**DML**),
- Data Control Language (**DCL**)
- Data Query Language (**DQL**)
- Transaction Control Language (**TCL**).

Data Definition Language (DDL)

- DDL or Data Definition Language actually consists of the SQL commands that can be used to defining, altering, and deleting database structures such as tables, indexes, and schemas.

Common DDL Commands

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure, and triggers)	`CREATE TABLE table_name (column1 data_type, column2 data_type, ...);`
DROP	Delete objects from the database	`DROP TABLE table_name;`
ALTER	Alter the structure of the database	`ALTER TABLE table_name ADD COLUMN column_name data_type;`
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	`TRUNCATE TABLE table_name;`
COMMENT	Add comments to the data dictionary	`COMMENT 'comment_text' ON TABLE table_name;`
RENAME	Rename an object existing in the database	`RENAME TABLE old_table_name TO new_table_name;`

SQL CREATE TABLE Statement

- To create a new table in the database, use the SQL CREATE TABLE statement. A table's structure, including column names, data types, and constraints like NOT NULL, PRIMARY KEY, and CHECK, are defined when it is created in SQL.

- The CREATE TABLE command is a crucial tool for database administration because of these limitations, which aid in ensuring data integrity.

- Syntax:

- To create a table in SQL, use this CREATE TABLE syntax:
- CREATE table table_name
- (
- Column1 datatype (size),
- column2 datatype (size),
- .
- .
- columnN datatype(size)
-);

- **table_name**: The name you assign to the new table.
- **column1, column2, …** : The names of the columns in the table.
- **datatype(size):** Defines the data type and size of each column.
- Here **table_name** is name of the table, **column** is the name of column

- Let us create a table to store data of Customers, so the table name is Customer, Columns are Name, Country, age, phone, and so on.
- CREATE TABLE Customer(CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), LastName VARCHAR(50), Country VARCHAR(50), Age INT CHECK (Age >= 0 AND Age <= 99), Phone int(10));

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
empty					

DROP and TRUNCATE in SQL

- **DROP Command**
- **DROP Table**
- **Syntax:**
- DROP TABLE table_name;
- **table_name**: Name of the table to be deleted.
- **DROP database**
- **Syntax:**
- DROP DATABASE database_name;
- **database_name**: Name of the database to be deleted.
- **TRUNCATE:**
- **Syntax:**
- TRUNCATE TABLE table_name;

- **DROP Database Example**
- In this example, we will drop the student_data database.
- **Query:**
- **DROP DATABASE** student_data; After running the above query whole database will be deleted.
- **DROP Table Example**
- In this example, we will drop the student_details table;
- **Query:**
- **DROP TABLE** student_details; After running the above query whole table from the database will be deleted.
- **TRUNCATE Table Example**
- In this example, we will truncate the Student_details table from the student_data database.
- **Query:**
- **TRUNCATE TABLE** Student_details;After running the above query Student_details table will be truncated, i.e, the data will be deleted but the structure will remain in the memory for further operations.

DROP	TRUNCATE
In the drop table data and its definition is deleted with their full structure.	It preserves the structure of the table for further use exist but deletes all the data.
Drop is used to eliminate existing complications and fewer complications in the whole database from the table.	Truncate is used to eliminate the tuples from the table.
Integrity constraints get removed in the DROP command.	Integrity constraint doesn't get removed in the Truncate command.
Since the structure does not exist, the View of the table does not exist in the Drop command.	Since the structure exists, the View of the table exists in the Truncate command.
Drop query frees the table space complications from memory.	This query does not free the table space from memory.
It is slow as there are so many complications compared to the TRUNCATE command.	It is fast as compared to the DROP command as there are fewer complications.

SQL ALTER TABLE

- The SQL ALTER TABLE statement is a powerful tool that allows you to **modify** the structure of an existing table in a database.
- The **ALTER TABLE statement in SQL** is used to add, remove, or modify columns in an existing table. The ALTER TABLE statement is also used to add and remove various constraints on existing tables.
- It allows for structural changes like adding new columns, modifying existing ones, deleting columns, and renaming columns within a table.

- **Syntax:**
- To alter/modify the table use the ALTER TABLE syntax:
- **AL**
- **1. ADD – To add a new column to the tableQuery:**
- ALTER TABLE table_name ADD column_name datatype;
- **2. MODIFY/ALTER – To change the data type of an existing column**
- ALTER TABLE table_name MODIFY COLUMN column_name datatype;
- **3. DROP – To delete an existing column from the table**
- ALTER TABLE table_name DROP COLUMN column_name;

- **4. RENAME COLUMN – To rename an existing column**
- ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
- **5. RENAME TO – To rename the table itself**
- ALTER TABLE table_name RENAME TO new_table_name;

SQL ALTER TABLE Queries

Suppose there is a student database:

ROLL_NO	NAME
1	Ram
2	Abhi
3	Rahul
4	Tanu

To ADD 2 columns AGE and COURSE to table Student.

Query:

```
ALTER TABLE Student ADD
(AGE number(3),COURSE varchar(40));
```

Output:

ROLL_NO	NAME	AGE	COURSE
1	Ram		
2	Abhi		
3	Rahul		
4	Tanu		

MODIFY column COURSE in table Student.

Query:

```
ALTER TABLE Student
MODIFY COURSE varchar(20);
```

After running the above query the maximum size of the Course Column is reduced to 20 from 40.

DROP column COURSE in table Student.

Query:

```
ALTER TABLE Student
DROP COLUMN COURSE;
```

Output:

ROLL_NO	NAME	AGE
1	Ram	
2	Abhi	
3	Rahul	
4	Tanu	

ALTER (RENAME) in SQL

Syntax

1. Renaming a Table

```
ALTER TABLE table_name
RENAME TO new_table_name;
```

2. Renaming a Column

```
ALTER TABLE table_name
RENAME COLUMN old_column_name TO new_column_name;
```

3. Adding a New Column

```
ALTER TABLE table_name
ADD column_name datatype;
```

4. Modifying a Column Data Type

```
ALTER TABLE table_name
MODIFY COLUMN column_name new_datatype;
```

Example 1: Rename a Column

Change the name of column name to **FIRST_NAME** in table Student. To change the column name of the existing table we have to use **Column keyword** before writing the existing column name to change

Syntax

```
ALTER TABLE Student RENAME COLUMN Column_NAME TO FIRST_NAME;
```

Query:

```
ALTER TABLE Student RENAME Column name TO FIRST_NAME;
```

Output

id	FIRST_NAME	age	email	phone
1	Shubham	23	shubham@gmail.com	9999999999
2	Bhavika	21	bhavika@yahoo.com	8888888888
3	Aman	21	aman@hotmail.com	7777777777
4	Sonia	23	sonia@gmail.com	6666666666
5	Kiran	19	kiran@yahoo.com	5555555555

Output

Example 2: Rename a Table

In this example, the table name `Student` is changed to `Student_Details` using the `ALTER TABLE` command, making the name more **descriptive** and relevant to its content.

Query:

```
ALTER TABLE Student RENAME TO Student_Details;
```

Output

id	FIRST_NAME	age	email	phone
1	Shubham	23	shubham@gmail.com	9999999999
2	Bhavika	21	bhavika@yahoo.com	8888888888
3	Aman	21	aman@hotmail.com	7777777777
4	Sonia	23	sonia@gmail.com	6666666666
5	Kiran	19	kiran@yahoo.com	5555555555

Student_Details table

SQL Comments

- **SQL Comments** explain sections of SQL statements or prevent SQL statements from being executed.
- There are 3 **types of comments** in SQL**:**
- **Single-line comments**
- **Multi-line comments**
- **In-line comments**

SQL Single Line Comments

- **SQL Single Line Comments** contain a single line a comment. They start and end in a single line.
- Single Line comments in SQL can be inserted using '–' before the line.
- **Syntax:**
- Syntax to insert single line comment in SQL is:
- — single line comment
- **SQL Single Line Comments Example**
- In this example, we use a single line comment to explain a query in SQL.
- **Query:**
- -- query to fetch customers records.
  SELECT * **FROM** customers;

SQL Multi-Line Comments

- **SQL Multi-line comments** contain multiple lines in a single comment. They start from one line and ends in a different line.
- A multi-line comment starts with '/*' and is terminated when '*/' is encountered.
- **Syntax**
- The syntax to insert a multi-line comment in SQL is:
- /* multi line comment
  another comment */
- **Multi-Line Comments Example**
- In this example, we will use the Multi-line comment to explain a complex query.
- **Query:**
- /*.
  The query selects all the orders from the orders
  table that were placed in the year 2022.
  */
  SELECT * **FROM** orders **WHERE** YEAR(order_date) = 2022;

SQL In-Line Comments

- **SQL In-line comments** are an extension of multi-line comments, these comments are used in between of a SQL statement.
- An In-Line comment starts with '/*' and end with '*/'.
- **Syntax**
- The syntax to use In-Line Comments in SQL is:
- SELECT * FROM /* Customers; */
- **SQL In-Line Comments Example**
- In this example, we are using In-Line comments in SQL to explain information about table.
- **Query:**
- **SELECT** customer_name,
  /* This column contains the name of
  the customer */ order_date / *
  This column contains the date the
  order was placed */ **FROM** orders;

Data Manipulation Language (DML) in SQL

- The **SQL commands** that deal with the **manipulation of data** present in the database belong to **DML** or Data Manipulation Language

Common DML Commands

Command	Description	Syntax
INSERT	Insert data into a table	`INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);`
UPDATE	Update existing data within a table	`UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;`
DELETE	Delete records from a database table	`DELETE FROM table_name WHERE condition;`

SQL INSERT INTO Statement

- The INSERT INTO statement is a fundamental **SQL command** used to add new rows of data into a table in a database.
- The **INSERT INTO** statement in **SQL** is used to add new rows of data to a table in a database.
- There are two main ways to use the INSERT INTO statement by specifying the columns and values explicitly or by inserting values for all columns without specifying them.

Syntax 1. Only Values

- The first method is to specify only the value of data to be inserted without the column names.
- INSERT INTO table_name VALUES (value1, value2, value);

Syntax 2. Column Names And Values Both

- *NSERT INTO table_name (column1, column2, column3) VALUES (value1, value2, value);*

Examples of SQL INSERT INTO

Examples of SQL INSERT INTO

For better understanding, let's look at the SQL Server INSERT statement with examples.

Let us first create a table named '**Student**'.

```
CREATE DATABASE StudentDB;
USE StudentDB;

CREATE TABLE Student (
    ROLL_NO INT PRIMARY KEY,
    NAME VARCHAR(50),
    ADDRESS VARCHAR(100),
    PHONE VARCHAR(15),
    AGE INT
);

INSERT INTO Student (ROLL_NO, NAME, ADDRESS, PHONE, AGE) VALUES
(1, 'Ram', 'Delhi', 'XXXXXXXXXX', 18),
(2, 'Ramesh', 'Gurgaon', 'XXXXXXXXXX', 18),
(3, 'Sujit', 'Rohtak', 'XXXXXXXXXX', 20),
(4, 'Suresh', 'Rohtak', 'XXXXXXXXXX', 18);
```

Created Table:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Ram	Delhi	xxxxxxxxxxxxxx	18
2	RAMESH	GURGAON	xxxxxxxxxxxxxx	18
3	SUJIT	ROHTAK	xxxxxxxxxxxxxx	20
4	SURESH	ROHTAK	xxxxxxxxxxxxxx	18

Example 1: Inserting Only New Values Using INSERT INTO Example
Example 2: Insert Values to Specified Columns Using INSERT INTO Example

- **Query:**
- **INSERT INTO** Student **VALUES** ('5','HARSH','WEST BENGAL', 'XXXXXXXXXX','19');

- **Query:**
- **INSERT INTO** Student (ROLL_NO, NAME, Age) **VALUES** ('5','PRATIK','19');

INSERT Multiple Rows in MS SQL Server

- **Syntax**
- **INSERT INTO** table_name(Column1,Column2,Column3,.....) **VALUES** (Value1, Value2,Value3,.....), (Value1, Value2,Value3,.....), (Value1, Value2,Value3,.....), ... ;

- **Example of Insert Multiple Rows in a table using Single SQL Statement**
- The following SQL statement inserts multiple rows in Student Table.
- **Query:**
- **INSERT INTO** Student (ROLL_NO, NAME, AGE, ADDRESS, PHONE) **VALUES** (6, 'Amit Kumar', 15, 'Delhi', 'XXXXXXXXXX'), (7, 'Gauri Rao', 18, 'Bangalore', 'XXXXXXXXXX'), (8, 'Manav Bhatt', 17, 'New Delhi', 'XXXXXXXXXX'), (9, 'Riya Kapoor', 10, 'Udaipur', 'XXXXXXXXXX');

SQL UPDATE Statement

- In SQL, the **UPDATE statement** is used to modify existing records in a table. Whether you need to update a single record or multiple rows at once, SQL provides the necessary functionality to make these changes

UPDATE Statement in SQL

- **Syntax:**
- The syntax for SQL UPDATE Statement is :
- UPDATE table_name SET column1 = value1, column2 = value2,…
  WHERE condition;

How to Use the SQL UPDATE Statement

Let's see the SQL update statement with examples.

First we will create a table, on which we will use the UPDATE Statement. To create the table, write the following query

Query:

```
CREATE TABLE Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    LastName VARCHAR(50),
    Country VARCHAR(50),
    Age int(2),
  Phone int(10)
);

-- Insert some sample data into the Customers table
INSERT INTO Customer (CustomerID, CustomerName, LastName, Country, Age, Phone)
VALUES (1, 'Shubham', 'Thakur', 'India','23','xxxxxxxxxx'),
       (2, 'Aman ', 'Chopra', 'Australia','21','xxxxxxxxxx'),
       (3, 'Naveen', 'Tulasi', 'Sri lanka','24','xxxxxxxxxx'),
       (4, 'Aditya', 'Arpan', 'Austria','21','xxxxxxxxxx'),
       (5, 'Nishant. Salchichas S.A.', 'Jain', 'Spain','22','xxxxxxxxxx');
```

The created table will look like this:

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Shubham	Thakur	India	23	xxxxxxxxxx
2	Aman	Chopra	Australia	21	xxxxxxxxxx
3	Naveen	Tulasi	Sri lanka	24	xxxxxxxxxx
4	Aditya	Arpan	Austria	21	xxxxxxxxxx
5	Nishant. Salchichas S.A.	Jain	Spain	22	xxxxxxxxxx

Example 1: Update Single Column Using UPDATE Statement

Update the column NAME and set the value to 'Nitin' in the rows where the Age is 22.

Query:

```
UPDATE Customer
SET CustomerName  = 'Nitin'
WHERE Age = 22;
```

Output:

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Shubham	Thakur	India	23	xxxxxxxxxx
2	Aman	Chopra	Australia	21	xxxxxxxxxx
3	Naveen	Tulasi	Sri lanka	24	xxxxxxxxxx
4	Aditya	Arpan	Austria	21	xxxxxxxxxx
5	Nitin	Jain	Spain	22	xxxxxxxxxx

Example 2: Updating Multiple Columns using UPDATE Statement

Update the columns NAME to 'Satyam' and Country to 'USA' where CustomerID is 1.

Query:

```
UPDATE Customer
SET
CustomerName = 'Satyam',
Country = 'USA'
WHERE CustomerID = 1;
```

Output:

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Satyam	Thakur	USA	23	xxxxxxxxxx
2	Aman	Chopra	Australia	21	xxxxxxxxxx
3	Naveen	Tulasi	Sri lanka	24	xxxxxxxxxx
4	Aditya	Arpan	Austria	21	xxxxxxxxxx
5	Nitin	Jain	Spain	22	xxxxxxxxxx

SQL DELETE Statement

- The **SQL DELETE** statement is one of the most commonly used **commands** in SQL (Structured Query Language). It allows you to remove one or more rows from the table depending on the situation.
- Unlike the **DROP statement**, which removes the entire table, the **DELETE statement** removes data (rows) from the table retaining only the table structure, constraints, and schema

SQL DELETE Statement

- **Syntax:**
- DELETE FROM table_name
- WHERE some_condition;

- **Note:** We can delete single as well as multiple records depending on the condition we provide in the WHERE clause. If we omit the WHERE clause then all of the records will be deleted and the table will be empty.

Examples of SQL DELETE Statement

Assume we have created a table named GFG_Employee in SQL which contains the personal details of the Employee including their id, name, email and department etc. as shown below –

```
CREATE TABLE GFG_Employees (
   id INT PRIMARY KEY,
   name VARCHAR (20) ,
   email VARCHAR (25),
   department VARCHAR(20)
);
INSERT INTO GFG_Employees (id, name, email, department) VALUES
(1, 'Jessie', 'jessie23@gmail.com', 'Development'),
(2, 'Praveen', 'praveen_dagger@yahoo.com', 'HR'),
(3, 'Bisa', 'dragonBall@gmail.com', 'Sales'),
(4, 'Rithvik', 'msvv@hotmail.com', 'IT'),
(5, 'Suraj', 'srjsunny@gmail.com', 'Quality Assurance'),
(6, 'Om', 'OmShukla@yahoo.com', 'IT'),
(7, 'Naruto', 'uzumaki@konoha.com', 'Development');
Select * From GFG_Employees
```

Output

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
4	Rithvik	msvv@hotmail.com	IT
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development

GFG_Employees

Example 1: Deleting Single Record

We can use the **DELETE statement** with a condition to delete a specific row from a table. The WHERE clause ensures only the intended record is removed. We can delete the records named Rithvik by using the below query:

Query:

```
DELETE FROM GFG_Employees WHERE NAME = 'Rithvik';
```

Output:

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development

Example 2: Deleting Multiple Records

Delete the rows from the table GFG_Employees where the department is "Development". This will delete 2 rows(the first row and the seventh row).

Query

```
DELETE FROM GFG_Employees
WHERE department = 'Development';
```

Output

id	name	email	department
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT

output

Example 3: Delete All of the Records

To remove all the entries from the table, you can use the following query:

Query:

```
DELETE FROM GFG_EMPLOyees;
Or
DELETE * FROM GFG_EMPLOyees;
```

Different Types of Database Users

- A Database User is defined as a person who interacts with data daily, up
- Database users can access and retrieve data from the database through the Database Management System (DBMS)dating, reading, and modifying the given data.

Types of Database Users

- Database users are categorized based on their interaction with the database.
- There are seven types of database users in DBMS. Below mentioned are the types of database users:

1. Database Administrator (DBA)

- A Database Administrator (DBA) is a person/team who defines the schema and also controls the 3 levels of the database.
- The DBA will then create a new account ID and password for the user if he/she needs to access the database.
- providing security to the database
- authorized users to access/modify the database.
- problems such as security breaches and poor system response time.
- recovery and backup
- technical support.
- system or superuser account.
- hardware and/or software failures.
- DBA is the one having privileges to perform DCL (Data Control Language) operations such as GRANT and REVOKE, to allow/restrict a particular user from accessing the database.

2. Naive / Parametric End User

- Parametric End Users are the unsophisticated who don’t have any DBMS knowledge but they frequently use the database applications in their daily life to get the desired results.
- For example, Railway’s ticket booking users are naive users. Clerks in any bank is a naive user

3. A System Analyst

- A system Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.

4. Sophisticated Users

- Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database.
- They can develop their own database applications according to their requirement.
- They don’t write the program code but they interact the database by writing SQL queries directly through the query processor

5. Database Designers

- Data Base Designers are the users who design the structure of database which includes tables, indexes, views, triggers, stored procedures and constraints
- controls what data must be stored and how the data items to be related
- It is the responsibility of Database Designers to understand the requirements of different user groups and then create a design which satisfies the need of all the user groups.

6. Application Programmers

- Application Programmers also referred as System Analysts or simply Software Engineers, are the back-end programmers who writes the code for the application programs.
- They are the computer professionals.
- These programs could be written in Programming languages such as Visual Basic, Developer, C, FORTRAN, COBOL etc.

Casual Users / Temporary Users

- Casual Users, also known as Temporary Users, interact with the database occasionally and for specific purposes.
- They usually require the database for tasks that do not occur on a daily basis but are crucial when needed.